软件工程 / 用户指南

ArquitecturaRepo

完整用户手册

版本 1.4 / 简体中文

全八章：简介、安装、项目接入、日常运行、变更请求、故障恢复、能力验收以及俱乐部实例。

MATÍAS ACUÑA · 2026年11月

依据ArquitecturaRepo公开源代码核对的用户文档。

目录

00 — 阅读本手册之前 6
本章内容 6
1. 一个可以复用的工程组织起点 6
2. 它比直接使用编程聊天更好吗 7
3. 角色与决定权 8
4. 事实的现在性与证据 9
5. 三个层面理解项目 10
6. 先采用适当的复杂度 10
7. 八章阅读路线 11
8. 哪些事必须由用户决定 11
9. 用会员管理系统贯穿学习 12
10. 判断采用的实际收益 12
来源与适用范围 13

01 — 安装和初始配置 14
本章内容 14
1. 准备开发环境 14
2. 三个容易混淆的位置 15
3. 安装前进行只读检查 16
4. 按风险选择配置 16
5. 为全新项目生成控制层 17
6. 生成成功后进行验证 18
7. 已治理项目必须保留历史 19
8. 常见问题与安全动作 19
来源与适用范围 20

02 — Planner、项目发现与正式接入 22
本章内容 22
1. 聊天记录不是完整的项目 22

2. 先调查工作空间，不急于写代码 23
3. Planner 应问哪些问题 24
4. 建立真实的需求与技术基准 24
5. 从初始 HOLD 过渡到正式接入 25
6. 让一个新会话证明可恢复性 26
7. 第一个能力如何获得授权 26
8. 会员管理的初次接入示例 27
来源与适用范围 28

03 — 日常运行：执行、测试与交接 29
本章内容 29
1. 开始前恢复当前项目事实 29
2. 一个能力应该有清楚的结果 30
3. I3 对普通技术失败拥有连续责任 30
4. 区分本地工程与敏感外部行为 31
5. 如何判断执行报告有价值 32
6. Git 和磁盘卫生也是工程连续性 32
7. 每日结束时回答五个问题 33
8. 会员登记的一天 34
来源与适用范围 34

04 — 提出变更请求与确定优先级 36
本章内容 36
1. 从用户遇到的问题出发 36
2. 一个可以直接使用的请求结构 37
3. 不要让 CLIENT 管理工程细节 38
4. 改变主意是正常的，但历史不能消失 38
5. 质量与可用性也属于真实要求 39
6. 先完成有价值的能力 39
7. 真正需要明确批准的情况 40
8. 五份具体可执行的产品请求 41
来源与适用范围 41

05 — 发生故障后的诊断与安全恢复 43
本章内容 43
1. 先保护证据，再改变系统 43
2. 本地环境和服务连接失败 44
3. 控制层未通过验证的正确解释 45
4. Git 不能代替数据库恢复 45
5. 重复出现的错误意味着需要改进方法 46
6. 敏感信息、访问控制和素材权利 46
7. 怎样确认故障已经真正结束 47
8. 三个恢复场景 48
来源与适用范围 49

06 — 能力验收与正确结束工作 50
本章内容 50
1. 能力是用户得到的结果 50
2. 五个要素构成一次可信的结束 51
3. 验收对象必须是同一个当前候选 51
4. 缺少证据不等于已经失败 52
5. 三种决策：接受、继续或重新定义 53
6. 反复返工需要正确的停止规则 53
7. 把决定写入能被恢复的位置 54
8. 会员登记候选的验收练习 54
来源与适用范围 55

07 — 完整示例：俱乐部会员和会费管理 56
本章内容 56
1. 一个规模不大但有实际风险的产品 56
2. 先让 CLIENT 确认业务含义 57
3. 在正确的目录进行安装 58
4. 从试验代码建立真实的基准 59
5. 第一项能力：登记与查看会员 59
6. 第一份候选为什么还不能验收 60
7. 第一项能力验收后才继续会费 61

8. 把缺陷交给正确的工程负责人 61
9. 用一个全新会话进行验收演习 62
来源与适用范围 63

00 — 阅读本手册之前

ArquitecturaRepo · v1.4 完整用户手册 · 简体中文

> 本章介绍 ArquitecturaRepo 的用途、适用边界、角色责任，以及它与普通编程对话的区别。内容针对公开的 v1.4.0 发行版，但不会声称某个采用者的真实应用已经完成或通过验收。实际命令以公开安装指南和发行说明为准。

本章内容

1. 这个仓库实际提供什么
2. 与直接在编程聊天中开发相比
3. 谁负责产品、战略与工程
4. 为什么证据比记忆重要
5. 用户意图、产品状态与实际代码
6. 从简单配置起步
7. 八章的阅读路线
8. 用户需要作出的决定
9. 会员管理的贯穿案例
10. 如何判断这个方法是否值得采用

1. 一个可以复用的工程组织起点

ArquitecturaRepo 是以 Git 仓库为中心的软件开发治理 starter。它提供职责与授权规则、可复制的模板、不同风险等级的配置、项目状态的组织方式、用于检查与恢复的 Python 工具，以及可选的参考运行机制。其目标是让新的开发者或新的代理会话能够从真实文件中恢复项目的当前状态，而不是要

求用户复述之前所有聊天记录。

它不是自动生成完整应用的产品。安装后并不会自动知道你的用户是谁、数据库位于哪里、哪项功能已发布、哪项操作已获授权。真实项目仍然需要需求确认、编码、测试、运行、数据保护和必要的人类验收。仓库提供组织与验证这些工作的结构，而不是替代工作本身。

也不需要先订阅某种大型语言模型或云端接口。你可以由人类团队执行这些流程；如果选择接入编程代理，必须另外核对代理的工作目录、权限、成本与使用边界。拥有工具访问权限并不等于获得发布、消费预算或上传私人数据的许可。

版本边界尤其重要。v1.4.0 的发布标签关联固定的 ZIP 包、SHA-256 校验值和清单；随后添加到 main 分支的完整手册不能被当作已经存在于原先的 ZIP 中。使用者应明确区分“已发布且可复现的发行物”和“持续更新的文档”。MIT 许可证和公开发布说明分别解释使用许可与版本范围。

2. 它比直接使用编程聊天更好吗

短小的实验里，直接告诉编程助手“写一个能够读取 CSV 的脚本”可能最快。你得到代码、运行一次、修改错误，任务就结束了。此时搭建复杂的治理层不一定有收益。我们不应仅凭文档页数或协议名称，就宣称 ArquitecturaRepo 在所有情况下更好。

但在需要持续数周、保存客户数据、多人参与的项目中，问题会不同。新的聊天可能不了解旧的决策；某次“测试通过”可能针对的是已经被改写的代码；开发者可能无法判断本次修改是否包括外部支付、生产数据库或公开发布。用户就不得不亲自调度技术细节、重复解释背景、区分哪些结果可信。

ArquitecturaRepo 希望把这些关系持久化：产品当前意图、批准的能力范围、负责任的角色、实际代码与数据库的标识、已运行的验证、仍然存在的风险、下一步被允许采取的行动。它

把 Git、规范化的项目状态与检查工具结合起来，让对话不再是唯一的事实来源。

评价方法应该是实证的：相同需求下，初次交付需要多久？修改需求后是否还能保持正确？新的开发者能否安全继续？问题出现时需要用户亲自排查多少内部技术细节？最终产品是否真正可用？这些结果比工具数量更有说服力。该架构能够改善可追溯性，但自身也带来维护状态和流程的成本。

3. 角色与决定权

CLIENT 是产品意图的负责人。需要解决什么问题、服务哪些用户、优先交付什么、什么体验可以接受，以及哪些外部操作或费用获得批准，都由有权的产品方决定。CLIENT 不必亲自决定每个类的名称、数据库迁移细节或 Git 冲突的处理。

角色	主要责任	不应承担的事情
I1 Master Owner（需要时）	多产品和共享资源的协调、系统性治理问题	为每个普通 bug 重建战略计划
I2 Planner	需求解释、技术策略、能力级计划、战略验收	把每次测试失败变成新项目
I3 Execution Controller	授权范围内的实现、调试、测试、恢复、复审和重做	擅自决定新的产品目标
I4 专项执行者或评审者	有边界的实现、调查、独立核查	越过授权代表 CLIENT 决策

四类责任不意味着四个人或四个 AI 进程。小项目可由少数人承担多项责任；当评价需要独立性时，必须注意实施者与评审者的利益和判断是否足够独立。让相同的代码作者在另一个窗

口里重复宣称 PASS，不等于独立审查。

Planner 设定能力的目的、范围和验收条件，获得授权之后由 I3 持续负责正常工程工作。普通的 bug、测试失败、内部重构与再次尝试，原则上在同一个 campaign 内解决。只有产品目标、授权边界或重要证据发生实质变化时，才重新讨论战略决定。

4. 事实的现在性与证据

真实的代码、运行配置、数据库、构建产物和验证结果，应当优先于旧报告和聊天总结。名为 CURRENT 的文件如果引用了已经不存在的提交，也不能因此自动成为真相。Git 提交的时间和作者信息同样不能单独证明当前的角色、授权或产品验收。

产品意图也会改变。如果 CLIENT 明确说“原来希望删除退出会员的记录，现在要保留付款历史”，新决策应更新当前需求，同时保留可追踪的历史。不能因为先前某段聊天写过另一种做法，就认定新决定无效。

当证据不足时，最安全的表达是“未验证”“不明确”“HOLD”，而不是填写虚构的成功标记。刚安装的控制层从 HOLD / UNTRUSTED_CONTEXT 开始，正是因为尚未核对真实项目的需求、代码、权限与状态。要解除这种不确定性，需调查并采用真实的基准，而非修改一个状态字符串。

如果数据库实际连接的是开发环境，而文档却称它为生产环境，不能继续执行依赖生产数据的操作。应先核对连接、状态和责任人。与此同时，与该不确定性无关的安全只读调查可以继续进行，不必把所有工作都永久停住。

5. 三个层面理解项目

可以把项目看作三个相互关联但不能混淆的层面。L0 是用户的表述和已确认的产品决定；L1 是整理成当前需求与 Product State 的产品模型；L2 是真实的代码、数据库、活动任务、测试和运行产物。

例如 L0 要求“保留每位会员的付款历史”。L1 应解释付款的含义、记录的生命周期以及可访问它的人。L2 则必须存在可靠的保存、查询和权限实现。仅仅在 L1 写了要求并不等于功能存在；L2 做出了某种保存功能，也不能自动证明满足 L0 的意图。

如果一个按钮显示“已支付”，需要问清楚它表示人工录入、外部服务确认，还是仅仅模拟的视觉效果。这三种情况的授权、风险、验收条件完全不同。软件可以在语法上正常运行，却在业务含义上完全错误。

新成员进入项目时，应当能够追踪三层之间的关系。只有聊天总结会丢失技术上的现在性；只有代码也未必能解释用户真正希望实现的结果。

6. 先采用适当的复杂度

架构的价值不是强制组建一个庞大的代理团队。一个简单的概念验证可以从 SMALL 配置开始；有持久会员信息的应用可考虑 STATEFUL；大量生成文件涉及来源和版权时可考虑 ARTIFACT_HEAVY。不同开发工作流协作、多个产品共享资源或高风险外部行为，对应其他配置。

先准备一张简短的产品说明：谁使用软件，最重要的操作是什么，要保存什么数据，运行在什么环境，第一版刻意不做什么，以及还有哪些问题未决定。这足以帮助 Planner 提出高价值的问题。用户不需要在开始时确定每个技术组件。

然后对真实代码和数据进行只读观察，建立可信的初始基准。首个交付可以很小，但应当是可运行、可验证、符合需求的真实能力。不要为了展示宏大结构而提前创建大量无人维护的状态文件。

7. 八章阅读路线

章节	主题	读完应当能做到
00 阅读前	概念、角色和事实	理解用途与局限
01 安装	获取、配置、验证	安全地创建控制层
02 项目接入	Planner 和需求	从真实应用建立基准
03 日常运行	实现、测试与复审	在同一授权范围持续工作
04 变更请求	用户反馈与优先级	清楚说明要解决的问题
05 故障恢复	保护证据与数据	排查原因并安全恢复
06 能力验收	区分质量关口	判断结果是否真正可用
07 俱乐部实例	完整实践路径	跟踪两个能力从提议到交付

需要立即入门时，可先看简体中文迷你指南与13个常见问题。这两份短文不是本书的替代品。本书对应 v1.4 的长篇内容，旧的西班牙语 v1.3 全书仍作为历史版本保存。

8. 哪些事必须由用户决定

软件最终要服务的工作流程、重要的用户体验、数据保留和访问政策、预算、外部发布等，可能需要 CLIENT 或其他有权人员做决定。技术人员应把选择的影响说明清楚，但不能把权限不足包装成“已经默认同意”。

如果用户说“让注册界面更好用”，工程团队可以提供设计方案与测试，但必须让真实用户评价是否更容易操作。如果用户说

“今天不要上线”，技术上的 release gate 即使通过，也不意味着可以擅自上线。

不需要为了每个内部技术错误都请求用户决策。实现者可以在授权范围内改进实现和测试。只有当产品含义、费用或外部风险变化时，再将需要用户决定的选项带回来。

9. 用会员管理系统贯穿学习

后续章节使用一个虚构的体育俱乐部。工作人员要登记会员、查询名单、记录会费；俱乐部负责人关心隐私和业务的连续性。第一个能力是会员登记与列表，第二个能力是人工录入会费记录。这些能力有依赖关系：会员标识必须可靠，才能将付款记录关联到正确的人。

我们不会假装已经开发出一个线上支付产品。案例只说明如何提问、安装、调查、授权、测试、发现问题、修复以及验收。真实项目需要真实的代码和用户反馈，才能证明功能已经存在。

10. 判断采用的实际收益

可以用两个规模相近的真实开发任务进行比较。记录从需求确认到可用版本的时间，后来修改需求所需的重做，跨会话交接时丢失的信息，问题出现时数据是否安全，以及用户是否必须亲自管理技术衔接。只有观察这些结果，才能判断治理层是否产生实际价值。

ArquitecturaRepo 适合强调长期连续性、责任边界和可恢复性的开发，但短期原型可能更适合直接编程聊天。让架构服务产品，而不是让产品成为架构的展示品。下一章从最容易出错的目录和版本问题开始，演示安全的安装路径。

下一章： 01 — 安装与初始配置。

来源与适用范围

- 公开安装与支持矩阵
- v1.4 发行说明
- 架构概览
- 角色与授权
- MIT 许可证

示例不代表任何采用者的应用已经实现、通过运行验证或获得发布许可。

01 — 安装和初始配置

ArquitecturaRepo · v1.4 完整用户手册 · 简体中文

> 这一章介绍如何识别正确的发行物、在正确的应用仓库中创建控制层、选择合适配置，以及理解为什么初次安装后的 HOLD 状态是安全的。请以官方 v1.4 安装说明为实际操作的依据。

本章内容

1. 准备工具与权限
2. 区分 starter、fork 和应用
3. 安装前的只读检查
4. 六种配置的选择
5. 生成 .agentic-sdlc 控制层
6. 安装后的检查
7. 已有治理状态的项目
8. 常见故障与安全处理

1. 准备开发环境

下列示例使用 Windows 和 PowerShell。需要能够访问 Git 仓库、运行 Python、在目标应用目录中拥有适当的文件权限。v1.4 的便携式持续集成测试使用 Python 3.12，但不要求某家专有云服务、商业 IDE 或特定 AI 模型。你可以选择自己的编辑器及合法的代码托管方式。

先查看当前终端环境：

```
& "python" --version
```

```
& "py" --version
& "git" --version
Get-Location
```

如果某个命令不存在，先检查安装位置、PATH 和终端会话。不要为了让检查通过而修改应用的 Product State。工具的可用性与产品需求的正确性是两回事。如果需要 Python 验证依赖，使用仓库提供的 requirements 文件安装，避免从不可信来源运行未知脚本。

使用编程代理之前，请确认其当前工作目录、可修改的文件范围、正在使用的 Git 分支以及是否有尚未提交的改动。即使代理已登录 GitHub，也不能据此推定它被授权操作生产数据库、创建付费资源或发布代码。相关授权要明确指向具体目标。

2. 三个容易混淆的位置

第一个位置是 ArquitecturaRepo 本身：里面存放公开协议、模板、配置、工具、测试和文档。第二个位置是你可能维护的 fork：它用于修改 starter 的通用规则，而不是应用产品。第三个位置是实际要开发的项目，例如 club-app：它拥有自己的代码、数据库以及安装后形成的 .agentic-sdlc/ 目录。

例如，一个清楚的本地目录安排是：

```
C:\Proyectos\
    ArquitecturaRepo\
    club-app\
```

通常从第一个目录运行生成器，把控制层写到第二个应用目录。不要打开 starter 本身，误以为正在开发俱乐部应用；也不要把 starter 自带的示例 Product State 当作真实会员需求。示例中的提交、测试、负责人和业务数据都是合成内容。

对于可复现安装，建议获取版本化的 v1.4.0 Release。它包含固定的 ZIP、SHA-256 校验与文件清单。验证下载文件后再使用。随着 main 分支持续更新，本手册这类后来增加的资料可能存在于源码树，但不属于原先 v1.4.0 ZIP 的内容。记录你使用的确切发行版本，而不只写“最新”。

3. 安装前进行只读检查

先进入实际应用的目录，查看原有代码、工作流、配置、数据库迁移与未提交改动。检查是否存在 .agentic-sdlc/。如果已经存在，那么这个项目可能已经有正式的状态、活动中的执行任务、授权记录或恢复依据，不能直接覆盖。

对于已有治理状态的应用，应该走已有项目的只读接入预检，而不是重复生成。尤其不要把 v1.2 或 v1.3 状态想当然地升级成 v1.4。迁移涉及旧决定、新格式和实际产品的数据，需要另行确认兼容性。

在新项目中，也先明确要解决的用户问题、保存哪些数据、希望部署在哪里，以及第一版明确不做什么。不知道的事项保留为未决。安装工具不知道你的产品细节，不能在生成时凭空替你确定。

如果一个目录已经包含大量应用文件，不需要为了配合教程而删除它们。这个 starter 的控制层应当附着于真实产品，而不是要求把真实产品重建成某个演示项目。

4. 按风险选择配置

配置定义位于profiles/profiles.json。应根据项目风险和实际运作方式选择，而不是根据想同时运行多少个代理来决定。

配置	适用特征
SMALL	简单、边界清楚，暂时没有复杂持久状态的产品
STATEFUL	持久数据、用户、备份、迁移与状态一致性重要
ARTIFACT_HEAVY	大量生成产物、存储成本、来源或版权重要
MULTI_WORKSTREAM	多个工作流共同组成同一最终产品
MULTI_PRODUCT	多个不同产品共享策略、人员或基础设施
HIGH_CONSEQUENCE	隐私、安全、金钱或不可逆操作可能造成重大影

对于学习用的小脚本，SMALL 通常足够。包含会员资料和会费历史的俱乐部应用可考虑 STATEFUL。配置越高并不代表产品越高级；增加不必要的管理流程可能延误真正的功能交付。选定配置后记录原因，在发现真实需求变化时调整。

任何配置都不能替代对生产环境的检查。名称为 STATEFUL 不代表数据库备份已实施；名称为 HIGH_CONSEQUENCE 也不代表安全审查已完成。所需证据取决于实际产品。

5. 为全新项目生成控制层

确认目标后，从 starter 的根目录执行以下 PowerShell 示例。命令是演示，不应对尚未检查的实际目录直接照搬：

```
& "python" "tools\scaffold_project.py" --
target "..\mi-proyecto" --profile "SMALL" --
product-id "MI_PROYECTO" --objective "Initial objectiv
-language "en" --no-feedback-workflow
```

虚构俱乐部的示例如下：

```
& "python" "tools\scaffold_project.py" --
target "..\club-app" --profile "STATEFUL" --
product-id "CLUB_MANAGER" --objective "Member and dues
-language "en" --no-feedback-workflow
```

将路径、标识和目标替换为你已经核对的实际值。这里的 –language en 是生成器的语言参数示例，不代表本手册的语言设置；如果安装脚本支持所需语言，可以按实际命令帮助进行选择。不要推测未测试的参数行为。

生成器创建的是目标项目内的 .agentic-sdlc/ 控制层，不是将应用功能自动实现。它应保护现有的规范控制层，拒绝普通的覆盖尝试。生成后看到 HOLD / UNTRUSTED_CONTEXT 是合理的：新的结构尚不知道真实代码、需求、所有者与当前执行状态。

可选的 –with-runtime 添加参考执行控制机制，但不会安装外部 LLM、启动原生 Codex Desktop 工作或替你批准某项任务。–no-feedback-workflow 关闭可选的定期反馈调度；本地元数据报告生成器是否启用由另外的配置控制。这里区分机制、权限和实际操作，避免误解。

6. 生成成功后进行验证

在实际应用的目录里，安装声明的验证依赖并运行状态、发现和恢复检查：

```
& "python" -m pip install -r ".agentic-
sdlc\requirements-validation.txt"
& "python" ".agentic-sdlc\tools\project_status.py" -
-control-root ".agentic-sdlc"
& "python" ".agentic-sdlc\tools\discover_project.py" -
-project-root "."
& "python" ".agentic-sdlc\tools\reconstruct_context.py
-root ".agentic-sdlc"
```

第一个脚本报告控制层状态；第二个检查实际项目中的文件与相关元数据；第三个尝试在没有旧聊天的情况下恢复现有文

脉。重建检查显示 HOLD 不一定意味着工具有 bug，也可能表示项目还没有完成正式接入。

确认管理目录位于预定目标、原有应用文件没有被覆盖、错误或未知信息可以从文件中读取。仅仅验证 JSON 格式与文件存在，并不能证明注册页面可以打开、数据库不会丢数据或用户能完成任务。

这一阶段的准确结果是“控制层已创建并检查”，不是“产品已采用”或“能力已验收”。接下来的正式接入需要调查真实代码、用户意图、产品状态和授权关系。

7. 已治理项目必须保留历史

如果 .agentic-sdlc/ 已存在，先确定它是否包含当前 Product State、批准的能力、执行租约、数据库引用、验证产物或恢复记录。覆盖或删除可能破坏未来唯一可用的恢复路径。

正确路线是查看已有治理项目接入说明、核对版本升级限制，必要时在只读模式建立兼容性报告。不要把新生成的空模板直接替换已经运行的治理状态，也不要把“重新安装一次”当作解决验证失败的默认办法。

对旧项目进行任何迁移，都必须先确认迁移的范围、当前的数据与代码、需要保留的历史、回滚方式以及是否获得适当授权。只有真正可追踪的证据能够支持迁移后状态的可靠性。

8. 常见问题与安全动作

看到的现象	可能意味着	应做的事
Python 命令不存在	本地环境未正确设置	检查安装和 PATH

看到的现象	可能意味着	应做的事
找不到 .agentic-sdlc/	目标路径错误或生成失败	核对命令日志和工作目录
jsonschema 缺失	验证依赖尚未安装	安装声明的 requirements
出现 HOLD	还缺真实项目的可信基准	继续调查与正式接入
提示已有控制层	项目可能已被治理	转入只读接入预检
Git 显示陌生改动	可能存在其他人的工作	识别所有者、保留改动
管理测试通过但应用无法使用	管理机制与物理功能不同	启动应用并检查用户旅程
外部账户没有权限	服务登录或授权不正确	核对范围，不泄露令牌

失败时记录原始错误、目标目录、工具版本和采取的动作。不要抹除日志、编造通过结果、强制清理整份仓库。正常的安装问题可以在原位置诊断与修复；治理范围和数据一致性问题需要更谨慎的处理。

下一章： 02 — Planner 如何发现真实产品并完成正式接入。

来源与适用范围

- 公开安装和支持矩阵
- v1.4 发行说明
- 首次 fork 与启动
- 项目接入与交接
- 已有治理项目的只读预检
- 验证机制
- 实际生成器代码

本章所有产品名称与命令参数仅用于说明，并非真实部署成功的证明。

02 — Planner、项目发现与正式接入

ArquitecturaRepo · v1.4 完整用户手册 · 简体中文

> 创建了 .agentic-sdlc/ 并不意味着项目已经被理解。真正的接入是将模板中的组织结构与实际产品的需求、代码、数据、权责和证据相连接。本章依据接入与交接规则以及产品发现说明解释这一过程。

本章内容

1. 为什么不能仅依靠聊天
2. 从只读调查开始
3. 向 CLIENT 询问真正影响产品的问题
4. 建立需求与技术基准
5. 从 HOLD 到可信的项目状态
6. 在全新会话中重新构建上下文
7. 第一个获得批准的能力
8. 俱乐部案例中的初次接入

1. 聊天记录不是完整的项目

用户在第一次对话中可能说：“我需要一个系统登记俱乐部会员。”这句话说明了方向，但不足以确定全部操作、数据保存规则、访问权限或实际的实现情况。助手若立即创建大型代码库，很容易把猜测写成事实；如果以后用户更改要求，又可能出现多份彼此矛盾的状态说明。

首先必须分清哪些是客户表达的愿望，哪些是经过确认的产品决定，哪些只是设计建议，哪些确实存在于代码中。如果之前的聊天说“登记 API 已经完成”，却没有对应当前提交、配置和

测试证据，不能把它当作已验证事实。

ArquitecturaRepo 的接入阶段正是为了避免这种混淆。HOLD / UNTRUSTED_CONTEXT 表示尚未证明模板的内容与真实项目一致。即使目录结构完好，仍需进行发现、确认、基准建立和可恢复性检查。将状态名称手动改成 READY，不会产生所缺少的事实。

2. 先调查工作空间，不急于写代码

Planner 应确认应用仓库的精确路径、当前分支、未提交变化、已有模块、数据库迁移和配置、测试入口以及可能运行的服务。最初尽可能使用只读方法，尤其是针对真实业务数据和既有治理状态的项目。

在已生成的项目控制层中，可使用下列工具辅助观察：

```
& "python" ".agentic-sdlc\tools\project_status.py" -
-control-root ".agentic-sdlc"
& "python" ".agentic-sdlc\tools\discover_project.py" -
-project-root "."
& "python" ".agentic-sdlc\tools\reconstruct_context.py
-root ".agentic-sdlc"
```

这些命令能够展示文件与状态方面的线索，但不能替代对业务含义的理解。看到一个名为 members 的数据库表，不能确定它用于正式会员还是测试样本；找到一个付款函数，也不能证明它执行实际收款。

如果已有 .agentic-sdlc/，先检查它是否包含当前权责、活跃 campaign、持久状态与恢复记录。此时应使用受治理项目的只读预检。不允许为了让当前教程“从头开始”而直接覆盖其他团队留下的控制信息。

调查需要区分开发与生产环境。一个在测试数据库中成功执行

的查询，不等于生产数据库已经通过当前性检查。未经允许，不应在真实会员数据库中批量写入示例记录。

3. Planner 应问哪些问题

好的问题能够改变产品的行为，而不只是改变技术名词。例如“会员可以同时参加两种活动吗”会影响业务模型；“谁可以修改会费记录”会影响授权与审计；“退出俱乐部后是否保留历史”会影响数据生命周期。

需要确认的问题	对系统的影响
会员由谁登记和查询	用户角色及访问控制
同名会员如何区分	稳定标识与重复检查
活动与会员是什么关系	数据模型与列表呈现
会费“已支付”代表什么	人工记录或真实支付的区分
退出后是否保留记录	删除、保存期限与隐私
一开始不做哪些功能	第一个 campaign 的边界

CLIENT 负责决定业务意图。Planner 可以提出现实可行的选项与利弊，不应要求 CLIENT 指定每个类或索引。技术团队负责解释实现方式、性能与维护成本；如果这些选择会实质改变产品目标、隐私或支出，应把决定返回有权的人。

对于尚未得到回答的问题，记录为明确的不确定性。如果“删除会员”可能表示软删除、完全移除或仅隐藏，那么这几种操作对应不同的风险，不能任选一种并声称用户已经批准。

4. 建立真实的需求与技术基准

调查完成后，把已确认的需求与具体的技术证据相关联。技术证据包括源代码位置与版本、数据库结构与环境、当前配置、

已执行的测试、已知故障、必要的运行产物。需要为每项重要能力说明：目前确实存在什么，尚未实现什么，哪些结果没有验证。

例如，会员列表页面已经实现，但新增会员只能通过直接修改数据库完成，那么“会员登记”这个能力仍然不完整。若注册 API 可以调用，但权限检查缺失，也不能直接认定真实的职员已具备安全的注册能力。

产品状态应使用明确的正本。决定的历史可以保留，但不要让两份文件同时声称自己是当前唯一的要求。Git 提交信息很有用，却无法单独证明用户验收、当前数据库的环境或者活动工作的责任人。每个关键事实都需要与其实际来源对应。

不必把项目所有未来想法一次性写进规范。先形成足够真实的初始模型，帮助下一位工程师识别正确目标和可安全行动的范围。过度详细、缺少证据的项目计划反而会产生虚假的确定性。

5. 从初始 HOLD 过渡到正式接入

接入可以理解为几个相互依赖的步骤。先观察真实应用与环境，再请 CLIENT 确认产品行为；之后确定技术基准、制定交付方向与接入说明，检查所有引用、责任与状态是否一致；最后将经过检查的状态正式写入规范位置，并重新验证可恢复性。

这一过程有专门的接入与交接合同约束。不能通过虚构 JSON、随便填一个提交哈希、凭空给出批准记录来让验证器变绿。无法确认的关键事实应保留为 HOLD，直到合适的人员提供证据或决定。

正式接入不意味着所有产品功能都完成。它表示管理层已经能够可靠地指向真实的产品，知道它当前处于什么状态，并且能够定义后续授权工作的范围。刚安装、已接入、正在实现、已经验收是四个不同里程碑。

如果发现已有项目治理状态与新版本不兼容，应先做只读兼容性检查，明确需要迁移什么、保护什么和如何恢复。不能自动删除旧记录，也不能未经授权“用新模板重建”。

6. 让一个新会话证明可恢复性

接入是否有效，最好的问题是：一个完全没读过旧聊天的工程师，能否从仓库中知道当前的产品目标、代码和数据库基准、已批准的能力、正在执行的 campaign、必要证据与下一步责任人？

reconstruct_context 是检查这个条件的工具之一。它的输出还应与实际仓库和状态对照。如果引用指向不存在的文件、过期的版本或另一个环境，就需要修正真实的根因。

想象原先的 I3 进程中断，新的执行者接手。新执行者应该知道尚未授权上线、只能修改会员登记模块、最新候选的测试情况以及如何恢复，而不是从上一次聊天的最后一句话猜测。如果做不到，项目管理状态仍缺乏可恢复性。

被标注为 CURRENT 的文本也需要检查是否与物理证据相符；不能因为名称包含“当前”就免于核对。正确的优先级是实际代码、数据库、运行产物和已确认的产品状态高于二手总结。

7. 第一个能力如何获得授权

接入之后，Planner 不必同时规划整套俱乐部信息系统的全部内部任务。可以把第一个可用结果定义为：“具有适当权限的职员能登记新会员并在列表中看到保存结果。”

这一能力的边界包括必要的输入字段、稳定标识、重复检查、权限和数据持久性；可以明确不包含会费、支付服务和活动预约。然后记录对代码、数据库与其他模块的依赖，提出可验证的验收条件和失败条件。

获得授权的能力级 charter 交给 I3 Execution Controller。I3 负责在范围内进行编码、内部设计、调试、迁移、测试、独立审查与重试。一个普通 bug 不应迫使整个项目重新走一轮 Planner 战略评估。

当用户要求外部在线支付或公开发布时，目标可能超出原有范围，涉及新成本与安全问题；此时需要新的决策。但如果只是注册按钮不能显示、API 返回错误或测试脚本有缺陷，应在原有授权内修复。

8. 会员管理的初次接入示例

假设读取现有 club-app 后发现：数据库中已有一个实验性的会员表，代码里有查询接口，但没有面向职员的注册界面。旧聊天曾声称“登记完成”，然而没有找到对应的端到端测试。Planner 应如实记录现状，而不是复制之前的完成声明。

与 CLIENT 讨论后，初版要保存姓名、联系信息、活动类型和内部编号；未经许可不能公开联系信息。无需在线付款。Planner 将这些决定及其来源写入当前需求，并把技术现状映射到代码与数据库的真实标识。

第一个 charter 可以要求登记后能够立即在列表中确认会员、重复编号被拒绝、普通访客不能写入、错误不会破坏现有数据。实现者自行决定技术细节，必要时提出有成本和风险的备选方案。

首次候选可能 API 测试通过，但浏览器的列表并不刷新。I3 需要在原有 campaign 内排查原因、修复 UI 更新并重新执行相关测试。用户真实使用后仍觉得完成状态不明确，也可以通过人类评价反馈同一能力的缺陷。

只有当前候选的代码、环境、数据库与验证结果可追踪，并且相关人员确认需要的用户旅程真正可用，才能验收该能力。验收记录存放在规范位置，使未来的新会话可以独立理解。

下一章： 03 — 进入日常工程工作后，如何在授权范围持续交

付并正确处理失败。

来源与适用范围

- 产品发现与需求、技术构成
- 产品发现会话指引
- 已存在项目的再锚定
- 接入与交接
- 已有治理项目的只读预检
- 角色与权责

本章的 club-app 是教学用的假设项目，并不代表任何实际应用已经接入或通过验收。

03 — 日常运行：执行、测试与交接

ArquitecturaRepo · v1.4 完整用户手册 · 简体中文

> 当项目已接入，某项能力得到批准后，工作进入持续工程阶段。本章介绍在新会话中恢复上下文、保持 I3 对普通技术问题的责任，以及如何通过真实的功能和证据评价进度。权责协议提供角色边界。

本章内容

1. 开始一天而不是从零开始
2. 从愿望得到具体可验证的能力
3. I3 持续负责哪些工作
4. 不能自行执行的外部操作
5. 怎样阅读完成报告
6. Git、并发和存储
7. 每日结束的五个问题
8. 一次完整的会员登记工作日

1. 开始前恢复当前项目事实

新的工程会话应先查看正确的应用仓库、当前分支、未提交改动、Product State、已经批准的 charter、活动中的执行任务与相关证据。聊天历史可以提示之前遇到的问题，但不能替代物理文件、运行状态和当前版本的核对。

例如，昨天的报告称“注册页面测试通过”，今天的分支却包含了未经核对的 UI 变更。不能直接沿用旧结论。应确认测试对应的提交、配置、DB 和实际页面。如果同一工作目录里有其他人的未提交文件，先识别归属与用途，而不是为了得到干净

状态就强制删除。

当来源互相矛盾时，先在只读模式调查差异。数据库不明的情况下，不允许执行依赖其状态的写操作；与问题无关的文档整理和本地安全测试则可能继续。正确的 HOLD 应有明确的范围和解除条件。

2. 一个能力应该有清楚的结果

用户说“把会籍管理做好”，含义仍然很宽泛。Planner 可以将首个能力限定为：“有权限的工作人员能够在网页上登记会员，看到保存后的列表，且不会因重复提交创建重复会员。”

定义范围时要写明输入、权限、数据保存、错误行为以及明确不包括的功能。例如实际收款、自动催缴、会员卡打印可暂时排除。这样工程团队能围绕一个完整的用户路径工作，而不是同时展开许多难以验收的支线。

能力级 charter 应说明为什么做、边界在哪里、依赖哪些系统、谁负责执行、如何验收、哪些现象足以否定完成。无需细化到“修改第几行代码”这种微型任务；内部实现路线应留给获得授权的工程负责人。

对简单产品，角色的数量也应保持适度。责任清楚比运行许多独立代理更有意义。如果同一人承担 Planner 与编码，需要特别留意重要验收时的独立性与利益冲突。

3. I3 对普通技术失败拥有连续责任

能力得到批准以后，I3 Execution Controller 负责实施、调试、重构、测试、恢复、评审协调和再次提交候选。数据库迁移在授权边界之内时，也属于这条工程链路。普通 bug 不应被当成新产品需求。

假设注册表单点击后页面不刷新。工程师应调查 API 返回、前端状态、缓存、数据库提交，修复根本原因并重新测试。无需要求 Planner 重新规划一个“修复刷新问题”的 campaign。若独立评审发现不符合原 charter 的问题，可把具体失败条件返回同一个执行链路。

如果同一类别的问题多次反复出现，不能无限追加局部修补和新编号。应系统性检查相关模块、依赖与测试方法，完成较全面的加固和独立审查。Planner 之后对合并后的终局结果进行战略性评价，而不是逐个批准每次内部调试。

但是，当工作需要连接真实支付网关、处理新的个人信息类型、改变部署方式或超出预设预算时，可能涉及产品与授权的实质改变。I3 不能仅凭现有 charter 自行扩大边界，应将具体影响提交到适当决策人。

4. 区分本地工程与敏感外部行为

读取代码、在授权目录中修改文件、执行本地单元测试，与向公网发布、使用付费接口、删除生产数据、把源代码发送第三方有不同的风险。后者必须明确目标、范围、权限和可恢复性。

一个已经登录的 GitHub 会话不等于获得对全部仓库的修改权。编程代理可以看到环境变量，也不意味着可以在回答中输出密钥。不得将客户数据、访问令牌、私人 URL 等复制到公开报告、问题单或本仓库。

对于自动化流程，先验证真实的运行边界。可选的参考 runtime 不保证原生 Codex Desktop、外部 AI 服务或第三方发布都已被测试。一次本地模拟成功，也不能成为对真实账户执行高风险操作的授权。

处理媒体和第三方依赖时，应满足实际适用的授权、隐私与许可证条件。缺少一张审计截图不自动证明素材违法；同样，有一个文件也不意味着已经取得全部权利。按真正的风险核实必

要信息。

5. 如何判断执行报告有价值

“完成 120 个测试”可以说明检查量，却不能回答用户能否真的登记一位会员。应让报告围绕具体能力：当前候选是什么、相对原有需求增加了什么、哪些质量关口通过、哪些尚未验证、是否需要用户实际操作确认。

质量关口	核心问题
Unit	单个函数或规则是否正确
Integration	API、数据库、界面是否能协作
DB / Currentness	检查对象是否是正确的当前数据与结构
Semantic	行为是否满足业务含义和异常情形
Physical	应用在实际环境里是否能启动和操作
Perceptual / Human	用户是否看得懂、用得顺手
Release	发行物、版本、发布权限是否正确

这些关口不能相互代替。绿色的单元测试不能证明网页已经打开；漂亮的演示也不能证明数据库安全；GitHub 上存在 ZIP 不代表它包含后续的文档改进。

审查应有合理的结束条件。真正影响能力的缺陷必须修复；与当前验收无关的微小警告不应导致无限期的战略停工。要让验收条件与实际风险决定投入，而不是通过不断制造审查轮次来衡量努力。

6. Git 和磁盘卫生也是工程连续性

Git 可帮助记录变化和准确引用代码，但提交本身不等于批准。一个分支的更新无法证明谁承担了工作、代码属于哪个

campaign、数据库对应哪个版本。重要决定仍需保存在合适的规范状态里。

多方同时开发时，按受影响的文件、数据库、服务和产物评估干扰，而不只是看它们是否属于不同项目。两个任务都修改相同配置就可能冲突；两个不同产品共享数据库资源也可能产生实际干扰。

通常只保留每个仓库一个持久工作副本。真正需要并发时，可暂时使用共享 Git 对象数据库的 worktree，完成后清理临时目录。大体积渲染、测试缓存、导出文件和中间产物应位于临时或被 Git 忽略的路径；只有必要的规范证据需要持久保存，并记录路径、哈希及来源。

“也许以后会用到”不足以成为无限复制仓库和产物的理由。磁盘耗尽会影响工程运行甚至恢复能力。清理时同样需要确定目标，不能误删当前的规范证据。

7. 每日结束时回答五个问题

结束一天之前，应当留下足够具体的信息，使新的会话能够继续。

1. 哪个用户能力取得了实际进展？
2. 证明这个进展的代码、配置、数据与产物分别是什么？
3. 哪些质量关口已经针对当前候选运行，哪些仍然未知？
4. 未解决问题的负责人是谁，是否仍属于原有授权？
5. 下一位工程师应从哪个规范文件与实际仓库位置开始？

不要把聊天的长篇逐字记录当成唯一的交接材料。只要用户意图、当前状态、技术证据和下一步路线能够从仓库恢复，就不需要重复过去几周的全部对话。

有时一天的主要成果是发现“之前以为有效的 DB 连接其实指向测试环境”。这种发现可能阻止了真实数据损坏，属于实质性的风险降低。与之相比，生成十份没有使用者的总结文件并不

一定算进度。

8. 会员登记的一天

早上，Controller 读取会员登记 charter 与昨天的候选状态。当前 API 测试已经通过，但用户点击保存后看不到新会员。团队先确认候选的代码与数据库版本，重现故障并发现前端保留了旧列表。

工程人员修复状态更新，重新运行单元与集成测试，然后启动真实界面完成登记和刷新操作。测试还覆盖重复点击按钮、无权限用户和错误后原始数据不被覆盖的情况。所有操作都发生在获批范围内。

下午，俱乐部职员试用表单，认为“保存成功”提示不够明显。Controller 可以围绕原能力优化提示，不必再次请求 Planner 创建一个新项目。假如职员又要求直接扣取会员银行账户里的会费，这就是新的外部金融行为，需要单独讨论。

最后，团队将具体候选的标识、验证结果、剩余限制、下一次操作与责任人记录在规范位置。第二天一个新会话打开时，应能找到这些信息，而不是只看到一句“昨天都弄好了”。

下一章： 04 — 用户如何清楚地提出变化，不需要亲自管理全部技术实现。

来源与适用范围

- 权责协议
- 参考运行机制
- 可执行验证
- 发现与交付设计
- v1.4 支持边界

会员登记是教学示例，不能据此声称某个真实系统已被实现或发布。

04 — 提出变更请求与确定优先级

ArquitecturaRepo · v1.4 完整用户手册 · 简体中文

> CLIENT 的任务是明确想要的产品结果，而不是替执行团队管理所有代码细节。本章介绍如何提出可验证的变更、处理意见变化，并且区分普通实现错误与真正的新范围。

本章内容

1. 先说结果，不急着指定代码
2. 一份简明的变更说明
3. 用户的决定和技术团队的责任
4. 改变想法时如何保留历史
5. 质量和使用体验
6. 不要同时打开十项工作
7. 哪些问题必须获得明确批准
8. 良好请求的具体示例

1. 从用户遇到的问题出发

“帮我给数据库增加一个索引”可能是技术方案，但并没有完整描述产品需求。用户真正想要的也许是：“工作人员能在已有 500 位会员的列表中迅速找到指定的人，不用等待或翻动许多页面。”有了具体目标，技术团队才可以调查瓶颈是索引、查询、缓存还是界面设计。

提出变更时尽量回答四个问题：谁遇到了什么困难？在什么环境或操作下发生？理想情况下应该看到什么结果？有哪些不能被破坏的限制？不必写长篇技术文档，但需要给出足够具体的

场景。

例如，前台工作人员需要通过姓名和会员编号查找记录。同名的两个人必须可区分；普通访客不应看到联系电话；退出俱乐部的会员应按照负责人确定的保留规则显示或隐藏。这些要求会影响产品的实际行为。

如果用户擅长编程，可以提出实现建议，但应与必须满足的产品结果分开。不要因为提议过一种技术方案，就不允许工程团队在有证据的情况下采用更安全、更简单的实现。

2. 一个可以直接使用的请求结构

不需要为每项需求新建复杂的战略文件。一份清楚的简明说明就足以启动调查。

项目	示例内容
目标	工作人员可以按姓名和编号搜索会员
使用者	俱乐部前台
当前问题	必须手动滚动很长的列表
期望结果	输入后显示匹配人，并能够识别同名会员
必须满足	无权限者不能看到私人联系方式
暂时不做	复杂统计和外部 CRM 同步
验收示例	两名同名会员可被正确区分
尚未决定	是否显示已退出俱乐部的会员

Planner 收到这份请求后，应检查当前应用、已批准的需求和正在进行的工作，判断是否可以并入原 charter，还是需要定义新的能力。只有影响真实产品方向的决定需要返回 CLIENT；测试脚本如何组织、使用哪种内部缓存则是工程工作。

“做得更专业一点”这种反馈并非没有价值，但要进一步描述具体问题。按钮不明显、输入错误后表单清空、移动端无法操

作、页面加载慢，都可以转化为用户旅程和验收条件。

3. 不要让 CLIENT 管理工程细节

CLIENT 负责产品定位、业务规则、价值排序和敏感授权。I2 Planner 负责战略分析、范围和能力级验收；I3 Controller 负责持续实现、调试、重做和验证；I4 可承担有限任务与独立评审。多产品共享治理需要时可以设 I1。

假如一个表单保存时抛出异常，I3 应找到根本原因、修复代码和检查回归；不需要向用户询问“这个异常该由哪一位代理处理”。如果数据迁移在原 charter 范围之内，技术团队也需要负责迁移设计与恢复测试。

但如果产品方后来要求“把所有会员信息公开给其他俱乐部”，这不是普通的内部代码修改。它改变了个人数据的使用范围和权限，需要有权的人评估并决定。技术实现不应通过惯性扩大授权。

一个好的系统让用户可以专注于产品内容与体验，同时使每个技术环节都有明确负责人。流程数量本身没有价值；能够持续交付和安全恢复才有价值。

4. 改变主意是正常的，但历史不能消失

一开始，用户可能说“退出俱乐部的会员直接从系统删除”，后来又说“过去的会费记录必须保留，前台只是看不到退出者”。新的明确决定可以取代旧要求，但之前的设计和已有的数据不能被简单抹除。

新的决定需要进入当前需求的规范位置。旧决定保留为可追踪的历史，并标明已被替代；技术团队检查现有代码、数据库、备份及迁移是否受影响。如果已经执行过不可逆的删除，不能仅修改界面就声称所有历史恢复了。

如果多份文件都以 CURRENT 名义陈述不同规则，新工程师可能依据错误的一份继续开发。应找到适合每项状态的唯一规范位置，避免通过复制摘要制造另一套互相冲突的真相。

同样，不应该把某次 Git 提交当成完整的产品批准记录。实际的决定者、受影响的能力、实施结果和当前性仍需要可追踪的证据。

5. 质量与可用性也属于真实要求

“功能存在但用起来难受”是有效的反馈。假设前台工作人员能够登记会员，但在保存成功后不知道系统是否完成了操作，于是反复点击按钮并创建重复记录。这个问题可能同时涉及交互设计、接口处理和数据完整性。

技术团队要区分不同的验证。API 单元测试可以说明某种输入能被处理；浏览器操作可以证明页面真的工作；真实使用者的观察可以判断完成状态是否容易理解。这些关口不能相互替代。

反馈可以包括屏幕、设备、操作步骤、预期与实际结果，但不要直接公开包含真实会员私人信息的截图。若需要复现问题，可以用合成数据和脱敏的日志。

对于性能，也应尽量说明测试条件，而不是只说“快一些”。例如：在普通职员电脑上，查询一定规模的会员数据时界面应在合理时间内响应。具体指标需要依据真实用户环境确认；文档中的示例数值不是所有项目的强制标准。

6. 先完成有价值的能力

用户可能同时提出搜索、统计、支付、通知、会员卡、移动端和外部同步。没有必要立即将它们拆成七个并发的 campaign。应优先考虑阻碍当前使用者的关键路径、共享依

赖、风险、成本和交付顺序。

例如先确保会员标识可靠，再开发会费关联。如果先做漂亮的会费报表，后来发现会员经常被错误合并，前期工作可能需要大量重做。优先级不仅由功能的表面吸引力决定，还取决于上游基础是否真实可用。

普通 bug 应在原有能力内修复。只有当用户确实改变目标、引入新风险或发现重大的战略证据时，才需要重新规划。不要为了提高“项目数”而让每次测试失败都变成新立项。

暂停某项工作时也要保留安全的停止点：当前候选、已实施变更、仍然存在的错误、需要恢复的环境和接手人。直接让新对话从零开始，会增加重复劳动与状态破坏的风险。

7. 真正需要明确批准的情况

“我们应该选择 PostgreSQL 还是 SQLite？”通常可以由技术人员根据并发、数据量、部署方式与成本提出建议。如果选择会显著影响产品预算或运维责任，可以让用户在清楚了解利弊后决定。

“能否把会员资料上传第三方？”“能否删除生产数据库里的旧记录？”“能否订阅付费 API 并开始扣费？”则直接涉及隐私、资金或不可逆操作。执行前需核对正确的授权者、范围和后果，不能把工具可用性当成许可。

如果某个决定仍未明确，受其影响的外部行为应当 HOLD。但是不依赖该决定的代码调查、原型演示或安全的本地测试，仍然可以继续。好的边界是停止有风险的部分，而不是让所有工程无意义地停摆。

8. 五份具体可执行的产品请求

以下示例都以产品结果为核心，并且留出了实现者的技术空间。

1. “我希望前台能通过姓名和会员编号搜索，结果要区分同名会员，未授权用户不显示联系电话。”
2. “保存新会员后要有清楚的确认提示；表单校验失败时，不要丢失用户已经输入的内容。”
3. “让我查看本月尚未缴纳会费的人员；第一版只记录人工收款，不连接在线支付平台。”
4. “会员登记页面在手机上也必须可操作，关键按钮的位置和提示应让实际工作人员试用后确定。”
5. “旧会员数据迁移到新结构时不能丢失，迁移之前需要验证备份和恢复方法。”

这些说明还不是最终工程 charter，但足以让 Planner 开始调查和分解。如果用户只说“全部使用 AI 自动化”，首先应问清楚哪一步需要自动化、是否涉及个人数据与外部服务、错误会产生什么影响、怎样判断自动化确实有帮助。

如果已完成的候选被用户拒绝，应具体指出与原有目的不一致的行为，交由 I3 在同一范围修复。若拒绝的原因是用户现在想要完全不同的产品，那才是新的战略议题。正确区分两者，既保护用户的决策权，也避免技术团队反复重开工作。

下一章： 05 — 出错时如何保留证据、保护数据库并排除根本原因。

来源与适用范围

- 产品发现、需求与交付
- 职责与授权
- 接入与交接

- 公开安装与使用限制
- 简体中文 FAQ

本章中的请求是教学例子，示例规模与响应时间不代表任何实际系统的测试结果。

05 — 发生故障后的诊断与安全恢复

ArquitecturaRepo · v1.4 完整用户手册 · 简体中文

> 真正的恢复不只是让测试重新显示绿色，而是找出原因、保护现有数据和证据，并在明确的范围内证明问题得到解决。本章区分环境失败、控制层错误、真实产品故障与敏感操作。

本章内容

1. 先保留现场和记录
2. 环境、依赖与账户问题
3. 控制层未通过验证
4. Git 不是一切故障的撤销按钮
5. 重复失败需要系统性修复
6. 安全、隐私和第三方素材
7. 如何结束一次故障处理
8. 三个具体恢复案例

1. 先保护证据，再改变系统

遇到失败时不要立即删除缓存、重建整个仓库或强制覆盖数据库。首先记录实际发生的现象、发生时间、执行操作、受影响的用户、当前代码与配置标识，以及合理可用的错误日志。必要时保留输入条件和可以复现问题的步骤。

如果日志包含会员姓名、联系方式、访问令牌等敏感信息，不要为了求助直接贴到公共聊天或 issue。可以通过合成数据重现问题，或者只记录经过适当隐藏的技术细节。保留证据不等于无限复制私人数据。

接着确认受影响的环境。是开发机上的验证脚本失败，还是生产系统无法登记会员？是一个页面问题，还是数据库可能损坏？恢复行动的优先级取决于真实风险。特别是当前生产数据不明的时候，不应让自动化程序未经授权进行破坏性写入。

对已有活动 campaign 的项目，还需要确认未提交的代码、当前执行租约、数据引用和最后可靠的候选。把这些记录删掉，可能使唯一的恢复路径消失。仅凭旧聊天说“之前已经备份”不足以证明备份仍然存在且可用。

2. 本地环境和服务连接失败

Python 不存在、jsonschema 未安装、工作目录错误、Git 凭据失效、网络服务暂时不可用，都可能导致检查失败。这类错误与产品需求本身正确与否不相同。

可以先在 PowerShell 中执行：

```
& "python" --version
& "git" status --short
Get-Location
```

如果缺少验证依赖，从安装的控制层所附的 requirements-validation.txt 中安装。不要修改检查器的预期结果来掩盖本地环境问题。也不要因 GitHub 暂时无法访问，就把本地仓库里的文件视为已经成功公开合并。

当代理连接到错误的仓库时，应立即停止写入，检查工作目录、目标分支、凭据的作用范围。具有登录会话并不代表获得跨仓库授权。如果问题涉及账户验证，应通过正规的登录和权限管理方式处理，切勿在聊天中索取或展示秘密信息。

3. 控制层未通过验证的正确解释

控制层检查可能报告缺少文件、无效的 JSON、引用断裂、状态矛盾、无效的权限或 CURRENT 与实际代码不一致。首先区分格式问题、环境问题、真实缺失的证据和某个验证器本身的错误。

首次生成的 .agentic-sdlc/ 返回 HOLD / UNTRUSTED_CONTEXT，可能是预期中的结果。它告诉你项目尚未完成正式接入，不应为了让仪表板变绿而填入虚构的当前提交、产品要求或验收记录。

如果测试器对一个真实有效的用例错误地报告失败，则需要检查协议和验证实现是否一致。这个技术问题可由 I3 在已授权的 campaign 内修复、复测并进行独立检查。无需每发现一个测试器 bug 就向 Planner 发起新的战略循环。

但如果真实数据库的版本无法确认，或者代码引用已经过期，仅修改测试器不能解决问题。必须重新核对代码、数据库、配置和当前状态之间的关系。证据缺失应被明确记录，直到找到可信依据。

4. Git 不能代替数据库恢复

有人看到测试失败后，可能想用强制 reset、删除目录、重新 clone 解决问题。这些操作有时会不可逆地丢失尚未提交的工作、审查证据或活动中的状态。执行之前必须确认确切目标、变更的归属和可恢复性。

Git 可以记录代码变化，却不能自动撤销生产数据库写入、支付服务调用或外部文件发布。即使把代码恢复到上周版本，如果上周之后的会员记录已经被删除，数据也不一定会回来。

涉及数据库迁移的恢复需要同时检查模式、数据、备份、迁移日志和应用的读写兼容性。在隔离环境中测试恢复方案，确认

关键记录之间的关系，再由有权的运维人员决定是否在真实环境执行。

有些问题可通过保留现有改动、修复一个文件和重新测试来解决。不要为了简单的拼写或编译错误而创建额外的完整仓库副本。恢复动作越接近真实问题，越容易解释和审查。

5. 重复出现的错误意味着需要改进方法

一次页面刷新失败可以由 I3 修复并重新验证。但如果相同模块连续多次返工，不能继续只增加局部判断和“修复第八轮”的文档。需要调查系统性原因：状态更新机制是否设计不当、API 契约是否含糊、测试是否根本没有覆盖实际页面、数据库环境是否经常混淆。

在这些情况下，I3 应针对整个相关子系统开展更完整的加固，检查依赖、异常路径、状态变化、并发条件与恢复方式，并且让独立评审者审视实现者的假设。完成之前，不应反复把短暂的测试绿色称为最终完成。

普通重做仍属于原有授权的工程工作。Planner 的下一次主要介入应是对已经整合的战略结果进行验收，而不是为每个小错误创建新 charter。只有当业务目标或实际风险条件发生重大变化时，才需要新的战略决定。

6. 敏感信息、访问控制和素材权利

故障调查常涉及日志、截图和数据导出。对于会员管理软件，这些材料可能含有真实的个人信息。调查只应收集必要范围，避免把完整客户数据库或私人聊天复制到公共仓库。访问令牌如有泄漏风险，应由有权的人员按照实际环境的安全程序轮换或撤销。

第三方图片、视频、字体和软件库的使用，需要遵守适用的许

可与授权。不能把“评审报告还没有附上许可截图”等同于“该素材一定禁止使用”，也不能仅因为素材能下载就认定可以无限制分发。应核对实际限制并保存必要来源。

本 starter 的反馈系统也有明确的隐私边界。v1.4 默认允许在采用者项目里产生最小化的本地元数据报告；是否周期性运行与是否向外部收集者发送，是不同配置。使用 –no-feedback-workflow 只处理调度，若要完全关闭生成器，应核对配置中的 enabled=false。反馈隐私文档说明这些区别。

连接任何外部 AI 或存储服务并不是故障修复的必需步骤。没有明确授权时，不应把私人代码或数据库导出给新的第三方。可以先利用本地检查、现有源代码和允许的工具进行诊断。

7. 怎样确认故障已经真正结束

恢复报告至少应当回答：故障的真实原因是什么？哪些代码、配置或数据发生了变化？问题是否在原条件下可重新验证？针对当前候选运行了哪些测试？是否已经检查了实际的产品操作？仍有什么风险？

不同的质量关口要分开报告。一个函数测试通过不等于 API 与 DB 集成正确；数据保存成功不等于页面可操作；开发环境正常也不等于生产环境已验证。需要时还应记录人工体验、隐私、安全和发布授权。

结束并不意味着证明世上所有情况都不会出错。应以与功能风险相称的检查为依据，说明测试覆盖的设备、数据和环境。对不影响当前验收的微小问题，可以公开记录并安排后续处理，而不必让项目无限停滞。

最后，把根因、修复、当前候选、验证、尚未完成的事项和下一个负责人写到可恢复的规范位置。一个新的会话应该能够理解障碍如何得到解决，而不是重新依赖原工程师的记忆。

8. 三个恢复场景

场景 A：新项目安装成功但仍然 HOLD

俱乐部项目刚生成 .agentic-sdlc/，状态报告指出真实需求、代码基准与所有者尚未采用。正确做法是继续项目发现和正式接入，而不是手动把状态改为 READY。

如果管理员为了让检查通过填写一个不存在的提交哈希，后续执行者就可能在错误的代码基础上继续工作。这个场景中的 HOLD 是有价值的保护，而不是需要隐藏的失败。

场景 B：页面显示“成功”，刷新后会员消失

前端表单展示成功信息，但重新打开列表找不到新会员。单元测试可能全部通过，因为它们没有检查真实的数据保存路径。I3 应调查接口返回、事务提交、连接的数据库和前端缓存，确认每一步是否指向相同的环境。

修复后在安全测试数据上完成注册、刷新、再次查询及错误恢复。若属于原本的会员登记能力，不需要新的 campaign。当前测试所对应的提交和数据库环境应被记录。

场景 C：一次迁移导致部分会费记录消失

发现历史会费缺失时，先停止可能扩大损害的写入，检查可用备份和迁移日志，比较当前数据与可恢复的基准。代码回退不能自动恢复缺失的数据。

若授权允许，可在隔离环境中验证备份是否完整，确认每笔会费仍对应正确的会员与月份。真正向生产环境恢复前，应评估停机、数据冲突、受影响人员和回滚方案。

恢复后检查的不只是“总金额看起来正确”，还应包括明细记录

与关联关系。只有实际数据完整性被合理验证，才能说明恢复达到了目标。

下一章： 06 — 对一个具体候选进行验收，区分可用的能力与漂亮的报告。

来源与适用范围

- 可执行验证
- 已有治理项目接入
- 隐私与反馈机制
- 角色和授权
- v1.4 使用边界

所有故障例子均为虚构，不能用于推断任何实际项目已经发生或解决这些故障。

06 — 能力验收与正确结束工作

ArquitecturaRepo · v1.4 完整用户手册 · 简体中文

> 一个工程任务报告“完成”，并不自动表示用户需要的能力可用。本章说明如何确定验收对象、区分不同验证关口、处理仍在原范围内的缺陷，以及形成可以让下一位工程师恢复的最终记录。

本章内容

1. 用户能力与代码任务的区别
2. 结束一项能力的五个要素
3. 验收的是哪个准确候选
4. 缺少证据时如何行动
5. 验收、继续修复或改变目标
6. 防止永无止境的审查循环
7. 留下可审核的验收记录
8. 会员登记的实际判断练习

1. 能力是用户得到的结果

假设客户希望“前台工作人员能够登记新会员”。开发者编写了保存数据的函数，这是一项实现工作，却不代表完整的产品能力。工作人员还需要有合适的界面、权限、输入校验、可理解的反馈和可靠的数据保存。

一项能力可以很小。把“能登记并查询会员”做成可靠的第一版，之后再增加会费记录，通常比同时声称拥有登记、自动支付、会员卡和分析报表更容易验证。每项能力应有清楚的边界，用户才能知道自己现在真正能够使用什么。

内部代码质量也重要。合理的重构、消除重复和更好的测试可降低维护风险，但它们应服务于用户需求。数十个提交或大量文档并不能自动替代“会员真的能被正确保存和查询”。

2. 五个要素构成一次可信的结束

可以从五个角度评价一项能力：

要素	要回答的问题
产品目标	当前结果是否满足已批准的用户需求
准确候选	哪个代码、配置、DB 与产物组合被检查
验证证据	必要的单元、集成、数据和运行检查是否完成
人类判断	需要时是否由实际使用者确认体验
交接连续性	结果、限制和下一步能否从仓库恢复

每个项目不需要完全相同的复杂流程。简单的命令行工具可能不需要视觉设计验收；处理会员个人信息的应用则不能忽略访问权限与数据保存。选择与真实风险对应的关口，而不是为流程本身增加流程。

形式化的验收条件最好在 charter 获得授权时就已经定义。如果工作完成后才临时增加大量与目标无关的要求，容易造成永无止境的差错搜寻；反过来，如果当初定义的关键风险尚未验证，也不能因为进度压力就把它们隐藏。

3. 验收对象必须是同一个当前候选

假设昨天的候选 V3 在测试数据库中运行正常，今天的候选 V4 改动了用户权限和数据库迁移。不能只引用 V3 的测试报告就批准 V4。应记录需要重跑的测试和实际的当前对象。

代码哈希有助于识别实现，但还需要确定配置、数据库版本、

相关资源与真实运行环境。某个截图可能来自旧代码，某个媒体产物可能被重新渲染，某个 CURRENT 文件可能指向已过期的版本。以精确的候选身份作为审查单位，才能避免不同版本的证据混合。

对数据库应用，尤其要检查“当前”是指开发环境、测试快照还是生产环境。开发快照里所有数据都一致，并不能单独证明生产 DB 已经适用。涉及外部服务时，还需要明确已调用的是模拟接口还是实际账户。

当候选发生改变，应基于影响范围决定需要重新验证哪些关口，而不是毫无证据地沿用全部旧 PASS。也不意味着每次修改都必须从头运行无关的大型测试；关键是让证据与改变后的对象保持对应。

4. 缺少证据不等于已经失败

如果单元测试通过，但没有任何实际页面操作记录，准确表述应是“单元检查通过，物理与人类体验未验证”。不能直接写“功能已全部完成”，也不能据此断定实际页面一定有故障。

缺少的检查属于原 charter 范围时，I3 应补充运行、记录或复查，并在同一 campaign 内提交新的候选。一次评审发现登记按钮失灵，或者 DB 校验脚本有 bug，不需要自动创建新的 Planner 项目。

需要区别三种情况：测试实际失败、测试尚未运行、测试曾运行但证据已经遗失。它们的恢复路线不同。测试失败要求修复原因；未运行要补充检查；证据遗失可能需要在相同条件下重新执行。只有真实记录支持的结论才应进入验收。

不过，若用户把原本只记录会费的功能改成从银行账户自动扣款，这是需求及外部风险的扩展。需要重新确认授权、费用和安全约束，不能当作普通的集成 bug 由 I3 自行决定。

5. 三种决策：接受、继续或重新定义

第一种是接受。当能力的实际用户路径符合产品目标，必要质量关口通过，版本与证据能被重建，相关人员已经确认使用体验和风险时，可以把能力记录为已验收。

第二种是继续在同一范围修复。如果仍有与原目标有关的bug、性能问题、集成缺陷或明显的人类体验不一致，就把具体失败条件交回原来的执行过程。I3 应继续修复并重新验证，而不是让 CLIENT 不断批准内部修补计划。

第三种是重新定义目标。只有在用户明确提出新方向、现有范围变得不合适、重要新事实改变了系统性风险时，才需要 Planner 与适当授权者重新审视能力的边界。

这三种情况都应该记录清楚。不能因为一位评审者写了“PASS”就取代用户授权，也不能因为一个无关的格式警告就永久搁置已经满足目的的产品能力。

6. 反复返工需要正确的停止规则

真正影响用户能力的缺陷不能被数量众多的绿色测试掩盖。如果同一模块连续出现同一类型失败，I3 应检查整个问题子系统，而不是只在每次报告里增加一个针对最新症状的修补。

可以从依赖、错误路径、并发、状态变化、测试夹具、真实运行环境和可恢复性等角度展开系统性加固，并进行足够独立的复审。完成后交给 Planner 一份整合的终局结果，而不是几十份彼此替代的临时结论。

另一方面，也应知道何时结束。如果能力要求已经满足，实质性的风险得到处理，仍然存在的轻微问题有明确记录和负责人，那么可以决定接受并进入下一项工作。完美的幻想不能成为永远不交付的理由。

验收应说明已经测试的环境与范围，而不是承诺“世界上任何

情况下都不会发生错误”。必要时通过有限用户、分阶段发布或回滚计划降低剩余风险，但发布本身仍要得到相应授权。

7. 把决定写入能被恢复的位置

最后的记录应包含：能力名称、对应的 charter、已验收的准确候选、验证结果与来源、批准者、仍然存在的限制、下一项能力的路线。把这些信息保存在项目的规范状态里，而不仅是某个聊天窗口的最后一条消息。

需要区分当前结果与历史。过去一次被拒绝的候选可以保留作为故障原因，但不应被误认为当前可发布版本。同样，曾经被接受的会员登记不能自动证明后来增加的会费功能也已通过验收。

下一位工程师应该能够独立找到正确的代码、数据库和所需证据，知道哪些能力已完成，哪些仍在进行，哪些操作尚未获批。这种可恢复性是长期项目的核心收益之一。

不要把接受记录无限复制到多个 CURRENT 文件。一个关切应有明确的规范所有者和当前表面。重复的“当前真相”越多，矛盾就越难发现。

8. 会员登记候选的验收练习

第一份候选实现了登记页面、API 和数据库写入。单元测试通过，但实际使用时工作人员连续两次点击保存，产生两名相同编号的会员。由于产品要求稳定、唯一的会员标识，此候选不能被接受。

I3 在原 campaign 中修复重复提交处理，并检查数据库约束、事务与并发情形。之后重新执行相关单元和集成测试，启动应用实际登记与刷新页面。还要验证无权限用户无法写入，以及保存失败不会破坏已有记录。

新候选表现正确后，前台工作人员仍发现成功提示不明显。工程团队改善操作反馈，再让实际使用者试用。当功能、数据安全与用户体验达到预先确认的条件，记录准确候选和证据，CLIENT 才能接受这一能力。

接下来开发会费记录时，不能引用“会员登记已通过”的结果声称支付系统也安全。新的能力需要其独立的范围、用户意义、数据关联与验收条件。对于任何真实在线支付，都还需要额外的授权与风控。

下一章： 07 — 把发现、安装、接入、两项能力与交接连成一个完整的俱乐部实例。

来源与适用范围

- 角色与授权
- 可执行验证
- 接入与交接
- 产品发现与需求
- v1.4 使用边界

这里的候选、测试和会员数据均为教学设定，不构成实际部署或业务验收的证据。

07 — 完整示例：俱乐部会员和会费管理

ArquitecturaRepo · v1.4 完整用户手册 · 简体中文

> 最后一章通过一个虚构的体育俱乐部，把用户初次表达需求、仓库安装、真实项目接入、授权的实现、反复修复、人工验收和最终交接联系起来。请注意：这是可复用的教学案例，不是已经发布的真实应用。

本章内容

1. 俱乐部的真实问题是什么
2. 做出会改变软件的业务决定
3. 在应用仓库中安装治理层
4. 调查并接入现有代码
5. 第一项能力：登记会员
6. 第一次候选为何不能通过
7. 第二项能力：记录会费
8. 把失败留在正确的工程范围
9. 使用新会话进行恢复演习

1. 一个规模不大但有实际风险的产品

假设一个社区体育俱乐部有前台工作人员、负责人和多种活动。当前成员信息散落在纸质记录与表格中。工作人员希望能快速登记新会员、找到现有会员、查询参加的活动，并查看会费情况。

对这个问题，第一反应可能是构建一套包含在线支付、会员卡、自动发送通知、复杂报表和智能预测的大平台。但用户眼

下最需要的，可能只是可靠的会员登记和手动会费记录。提前扩展会增加开发与验收成本，也可能引入未得到授权的外部数据服务。

因此先确认两个能力。第一项让有权限的职员登记并查询会员；第二项让职员为已登记会员录入会费支付记录并查询未缴情况。本案例不执行真正的在线扣款，也不自动公开任何私人数据。

因为系统要持久保存姓名、联系方式和缴费历史，可以选择 STATEFUL 配置。这个选择意味着需要认真考虑数据一致性与恢复，但不会自动证明数据库已有备份，也不强制建立一个庞大的代理组织。

2. 先让 CLIENT 确认业务含义

Planner 不应从“会员管理”四个字自动推导所有细节。至少需要确认会员如何唯一识别、谁可以登记和编辑、会籍退出如何处理、每项活动是否可能有不同费用、会费缴纳记录与真实银行交易是否关联。

事项	本案例的教学假设
会员标识	每位会员有稳定且唯一的内部编号
必需信息	姓名、联系方式、活动类型
操作者	有授权的前台职员
可见性	未经许可不显示私人联系方式
退出会员	保留历史的具体期限仍需确认
会费	手动记录收到款项，不发起银行交易
首版排除	在线支付、CRM 同步、自动通知、复杂分析

实际部署时，这些假设必须由真正的俱乐部负责人确认。尤其是删除与保留历史的规则，不能由工程师根据方便程度擅自决定。没有得到回答的部分可以暂时标记为未定，并避免执行依

赖该决定的不可逆行为。

优先解决哪些问题也由产品价值决定。若人员根本无法准确登记会员，那么先开发漂亮的财务图表不能解决核心困难。需求应尽可能反映使用者完成工作的顺序。

3. 在正确的目录进行安装

开发者把公开的 ArquitecturaRepo starter 与实际的 club-app 分开。需要复现发行内容时，从固定的 v1.4.0 Release取得 ZIP 并核对 SHA-256。持续变化的 main 分支包含发行后的文档，不等于过去的发行包。

在确认目标目录是全新、且没有现有 .agentic-sdlc/ 后，可以从 starter 的目录运行示例命令：

```
&  "python"  "tools\scaffold_project.py"  --
target  "..\club-app"  --profile  "STATEFUL"  --
product-id "CLUB_MANAGER" --objective "Club member and
-language "en" --no-feedback-workflow
```

目标目录、ID 和说明应换成实际项目的信息。如果 club-app 已经具有治理状态，应该转向已有项目的只读接入预检，而不是强制覆盖。

生成后检查实际应用是否还完整，运行必要的状态工具：

```
& "python" ".agentic-sdlc\tools\project_status.py" -
-control-root ".agentic-sdlc"
& "python" ".agentic-sdlc\tools\discover_project.py" -
-project-root "."
& "python" ".agentic-sdlc\tools\reconstruct_context.py
-root ".agentic-sdlc"
```

最初显示 HOLD / UNTRUSTED_CONTEXT 不代表工作失败。这个结果提醒我们：模板无法凭空知道真实产品的代码、DB、需求和授权。接下来仍需要 Planner 进行调查与正式接入。

4. 从试验代码建立真实的基准

假设 club-app 中已经有一个试验性会员表，支持按 ID 查询，但没有可供职员使用的登记表单。旧聊天提到“登记功能已完成”，可是在当前仓库里没有对应页面和最新测试。Planner 应相信实际代码与运行证据，而不是旧报告。

先以只读方式确认数据库环境、表结构、现有 API、代码提交、活动分支以及可能尚未提交的改动。对于真正的会员数据，必须避免未经授权的修改与导出。找到真实证据后，将其与 CLIENT 已确认的产品意图对应起来。

在接入与交接规则下，Planner 建立清楚的需求、当前技术基准和责任边界。采用规范化的状态记录，而不是把合成样例中不存在的提交哈希和“测试通过”声明复制过来。

接入完成的意义是：新工程师能够知道真实项目位于哪里、产品要做什么、目前已经实现什么以及下一项授权工作是什么。它尚未等于登记功能已经完成。

5. 第一项能力：登记与查看会员

Planner 将最初的能力定义为：有权限的职员输入姓名、联系方式和活动类型，保存会员记录，并在列表中看到结果。相同编号不可重复；错误发生时不会损坏已有会员；无权限的普通访客不能登记。

需要的依赖包括会员 ID 的规则、数据库保存、API 与界面之间的协议、访问控制以及最基本的异常处理。故意排除会费与支付，有助于让能力清楚且可验证。

授权的 charter 描述目标、范围、依赖、责任人、验收条件和失败条件。它不需要指定每个类与实现步骤。I3 Execution Controller 接手后，负责内部技术方案、实现、测试、调试、恢复和独立复审。

如果工程团队使用临时模拟数据做演示，应明确这一点。不能把演示页面与实际有持久保存功能的产品混为一谈。完成代码编写只是第一步，后面还需要多层验证。

6. 第一份候选为什么还不能验收

假设第一个候选 API 测试通过，保存后返回正常状态，但职员在网页上看不到刚刚登记的会员。开发者可能觉得“后端已经完成”，然而用户需要的是完整的登记路径。对于这个能力，实际界面不刷新是实质性缺陷。

I3 调查前端状态、接口响应、缓存和数据库连接，在原 campaign 中修复。修复后重新运行单元、集成与浏览器测试，检查保存后列表更新以及重新打开系统后数据是否还在。

第二个问题是按钮连续点击两次会创建重复会员。Controller 应调查接口幂等性、数据库唯一约束与并发请求，而不是只依靠前端阻止第二次点击。对真实业务而言，重复会员可能使后续会费错配，因此属于必须验证的行为。

第三个问题来自职员的实际体验：界面虽然正确保存，但成功提示不明显。工程团队应听取用户反馈，改善操作提示。这是对人类使用质量的检查，不能被单元测试自动取代。

如果这些问题都属于原授权的登记能力，就应在同一 campaign 内修复、整合和复审。无需让 Planner 针对每次 bug 重新设计整个产品。只有业务边界真的改变时才要重新决定。

7. 第一项能力验收后才继续会费

针对当前准确候选，技术团队验证权限、数据保存、重复处理、实际页面操作，职员也确认可以理解并使用。然后在规范位置记录已经验收的代码和配置、相关 DB 证据、实际测试、批准者、剩余限制及下一步工作。

接下来 Planner 定义第二项能力：工作人员可以为会员录入某月的会费支付记录，并查询未缴情况。它依赖第一项能力的稳定会员 ID。如果会员可以被错误合并或随意重用编号，账目会与错误的人关联。

要再次确认会费周期、金额、补缴、作废、退款或更正的业务含义，以及谁有权限修改旧记录。这里仍然只做人工会费记录，不代表已经连接银行或支付平台。

第二项能力需要自己的测试：同一个月重复录入、错误的会员编号、权限不足、网络失败后重试、月份边界、数据持久性，以及历史记录不能被意外覆盖。第一项能力的“登记通过”不能替代这些会费相关的验证。

如果后来用户希望真正在线收款，需要考虑合同、资金流、隐私、外部成本和事故恢复。这是产品范围的重大变化，应先获得适当授权，而不是由 I3 在原有的手动记录 charter 中自行添加。

8. 把缺陷交给正确的工程负责人

假设会费记录候选在测试 DB 成功，但实际界面把上个月的未缴名单显示为本月。首先应核对日期与时区规则、查询过滤、缓存、数据基准和实际使用情形。这个问题在已授权的会费记录能力之内，I3 应继续处理。

若同一类日期错误反复出现，团队不应只修补某个条件分支。应系统审查时间和月份的业务定义、数据库字段、测试夹具、

不同的边界日期与真实查询路径，再让独立评审者检查。没有重要的新战略证据时，不需要创建多个替代性 campaign。

反过来，如果调查发现实际俱乐部属于多个国家，须支持不同货币与不同财务规则，原有设计的业务范围可能已经不适合。这属于会改变系统重要约束的新事实，应提交 Planner 与 CLIENT 重新确认。

每次结果都要说明准确的代码、数据库和环境。来自演示数据的 PASS 不代表生产数据的当前性已经核对；来自旧候选的截图也不代表当前的界面仍然正确。

9. 用一个全新会话进行验收演习

假设最初的所有聊天都不可用。打开同一仓库的新工程师应该能够回答以下问题：

1. 俱乐部软件当前为哪些人服务，CLIENT 批准了什么目标？
2. 会员登记能力已经验收的是哪个具体候选？
3. 会费记录能力处于未开始、进行中还是已验收状态？
4. 当前代码、数据库、配置和验证证据分别在哪里？
5. 哪些问题仍然属于已批准的 I3 工作，谁负责继续？
6. 哪些行为，如在线付款与公开发布，尚未获得授权？

如果无法回答，就说明仍缺乏可信的交接信息。应补齐正确的规范记录、修复断开的引用、核对当前技术来源，而不是让新的代理通过猜测填补空白。

最后，可以把这个开发过程与直接在编程聊天中开发相比较。ArquitecturaRepo 并不保证短期原型更快，但如果它让需求变化、跨会话恢复、错误复查与真实验收更可靠，并减少用户亲自组织技术工作的负担，就产生了可测量的价值。

全书结束。 若只需快速入门，请阅读简体中文迷你指南与常见问题。对于实际安装，始终优先核对v1.4 公开指南、所用的发

行物以及真实应用的状态。

来源与适用范围

- 配置文件
- 项目生成器
- 接入与交接
- 发现、需求与交付
- 角色与授权
- 实际验证机制
- 公开 v1.4 安装说明

本章仅为贯穿式教学案例，未生成真实 club-app，亦未代表任何采用者完成验收或发布。